INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-04: Clases, Objetos y Métodos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primer Semestre – “B”
Alumnos:	Alulema Chato Sandra Jazmin
	Armas Jaya Anthony Daniel
	Guaman Sagñay Joselyn Maribel
	Ortiz Mosquera Valeria Valentina
	Vayas Masache Andrés Gabriel
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. José Rubén Caiza Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

- Desarrollar programas en Java utilizando clases, objetos y métodos, lo cual permite disminuir errores en el código y reutilizarlo.

Específicos:

- Definir métodos con el tipo de retorno, los parámetros y los argumentos adecuados, diferenciando el uso de `void` y de `return` y comprendiendo el paso por valor.
- Crear clases con atributos y métodos, instanciar objetos y hacer que varias clases colaboren entre sí mediante objetos como atributos y como parámetros.
- Aplicar estructuras condicionales y repetitivas (`if`, `else`, `while`) para validar datos y resolver problemas de inventario, cuentas, notas y biblioteca.

2.2 Instrucciones

- Realizar la tarea de forma individual o según las indicaciones del docente.
- Leer detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrollar lo solicitado.
- Codificar en Java cada uno de los ejercicios planteados y comentar el código.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado y adjuntar las capturas de cada una de las clases y métodos en un archivo PDF con el formato de Guías APE.
- Presentar el trabajo completo, ordenado y dentro de la fecha establecida.

2.3 Listado de equipos, materiales y recursos

- Inteligencia artificial.
- Computador o laptop.
- Plataforma educativa de la UTA.
- Visual Studio Code.
- Java (JDK).

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒ Plataformas educativas
☐ Simuladores y laboratorios virtuales
☒ Aplicaciones educativas
☐ Recursos audiovisuales
☐ Gamificación

☒ Inteligencia Artificial
Otros (Especifique): _____

2.4 Actividades por desarrollar

➢ **Enunciado:** Desarrollar en Java los ejercicios de la APE 4, organizados en tres partes: (A) métodos, parámetros, `void` vs. `return` y paso por valor; (B) integración de clases, objetos y métodos; y (C) introducción a la programación orientada a objetos. Cada ejercicio incluye su código y la salida real de la ejecución.

➢ **Análisis del problema:**

o Datos de entrada: valores asignados a los atributos de los objetos (nombres, notas, precios, stock, saldos, edades) y argumentos enviados a los métodos.

o Procesos: definición de clases y métodos; creación de objetos con `new`; llamadas a métodos con parámetros; cálculos (promedios, totales, transferencias); decisiones con `if / else if / else` y repeticiones con `while`.

o Datos de salida: resultados y mensajes impresos en consola (totales, estados «Aprobado / Reponer / Prestado», mensajes de validación y de error).

PARTE A. Métodos, parámetros, void vs. return y paso por valor

Ejercicio A1. Identificación de tipos de retorno

Enunciado: Dado el siguiente método, explicar por qué genera un error de compilación en Java y cómo se corregiría.

```java
public int sumar(int a, int b) {
    System.out.println(a + b);
}
```

Archivo: Main.java

```java
public class Main {

    // Version con error de compilacion (no se puede compilar):
    // public int sumar(int a, int b) {
    //     System.out.println(a + b);
    // }
    // Error: el metodo declara retorno int pero no tiene return.

    // Correccion 1: agregar return con el tipo declarado
    public static int sumar(int a, int b) {
        return a + b;
    }

    // Correccion 2: si solo imprime, cambiar el tipo de retorno a void
    public static void mostrarSuma(int a, int b) {
        System.out.println("Suma: " + (a + b));
    }

    public static void main(String[] args) {
        int resultado = sumar(5, 3);
        System.out.println("Resultado con return: " + resultado);
        mostrarSuma(5, 3);
    }
}
```

Explicación: El método declara que retorna un `int`, pero no tiene ninguna instrucción `return`. Java exige que todo método con tipo de retorno distinto de `void` devuelva un valor del tipo declarado. La corrección 1 agrega `return a + b;`; la corrección 2 cambia el tipo de retorno a `void` cuando el método solo debe imprimir.

Salida de la ejecución

```
Resultado con return: 8
Suma: 8
```

Ejercicio A2. Métodos void para acciones directas

Enunciado: Crear dentro de la clase `Bienvenida` un método que reciba un nombre (`String`) y una edad (`int`) e imprima: «Hola [nombre], tienes [edad] años. ¡Bienvenido al sistema!». ¿Qué tipo de retorno debe llevar y por qué?

Archivo: Bienvenida.java

```java
public class Bienvenida {

    // void porque solo imprime en consola y no devuelve ningun dato
    public void saludar(String nombre, int edad) {
        System.out.println("Hola " + nombre + ", tienes " + edad + " años. ¡Bienvenido al
sistema!");
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Bienvenida b = new Bienvenida();
        b.saludar("Bruno", 20);
    }
}
```

Explicación: Debe llevar `void`, porque el método realiza una acción directa (imprimir en consola) y ningún otro bloque de código necesita un resultado.

Salida de la ejecución

```
Hola Bruno, tienes 20 años. ¡Bienvenido al sistema!
```

Ejercicio A3. Parámetros vs. argumentos

Enunciado: En el siguiente código, identificar cuáles son los parámetros y cuáles los argumentos, explicando la diferencia entre ambos.

Archivo: Main.java

```java
public class Main {

    // a y b son PARAMETROS: variables declaradas en la firma del metodo
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        // 5 y 3 son ARGUMENTOS: valores reales enviados al llamar el metodo
        int resultado = sumar(5, 3);
        System.out.println("Resultado: " + resultado);
    }
}
```

Respuesta: Parámetros: `a` y `b`, variables declaradas en la firma del método. **Argumentos:** `5` y `3`, valores reales que se envían al llamar el método. El parámetro es el molde que recibe el dato dentro del método y el argumento es el dato real que se envía en la llamada.

Salida de la ejecución

```
Resultado: 8
```

Ejercicio A4. El comportamiento del paso por valor

Enunciado: Si la variable `numero` del `main` vale 10 antes de llamar al método, ¿cuál será su valor impreso después de ejecutarlo? Explicar por qué.

Archivo: Main.java

```java
public class Main {

    public static void modificar(int numero) {
        numero = numero + 5;
        System.out.println("Dentro del metodo: " + numero);
    }

    public static void main(String[] args) {
        int numero = 10;
        modificar(numero);
        // Imprime 10: Java usa paso por valor, el metodo trabaja con una copia
        System.out.println("Despues del metodo: " + numero);
    }
}
```

Respuesta: El valor impreso será **10**. Java usa paso por valor: el método recibe una copia de la variable, por lo que el cambio dentro del método afecta solo a la copia y no a la variable original del `main`. Dentro del método se imprime 15 y después del método se imprime 10.

Salida de la ejecución

```
Dentro del metodo: 15
Despues del metodo: 10
```

Ejercicio A5. Método con múltiples parámetros

Enunciado: Agregar a la clase `Calculadora` el método `calcularPromedio`, que reciba tres notas `double` (n1, n2, n3), las sume, divida entre 3 y retorne el resultado.

Archivo: Calculadora.java

```java
public class Calculadora {

    public double calcularPromedio(double n1, double n2, double n3) {
        return (n1 + n2 + n3) / 3;
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        double promedio = calc.calcularPromedio(14.5, 16.0, 18.5);
        System.out.println("Promedio: " + promedio);
    }
}
```

Explicación: El método retorna `double` porque entrega un valor calculado que el `main` guarda en la variable `promedio`. Se envían tres argumentos que coinciden en número, orden y tipo con los parámetros.

Salida de la ejecución

```
Promedio: 16.333333333333332
```

Ejercicio A6. Validación de datos (control de errores)

Enunciado: Crear el método `verificarEdad(int edad)`. Si la edad es menor a 0 o mayor a 120, imprime «Edad inválida» y retorna `false`; si es correcta, retorna `true`.

Archivo: Validador.java

```java
public class Validador {

    public boolean verificarEdad(int edad) {
        if (edad < 0 || edad > 120) {
            System.out.println("Edad inválida");
            return false;
        }
        return true;
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Validador v = new Validador();
        System.out.println("Edad 25: " + v.verificarEdad(25));
        System.out.println("Edad -3: " + v.verificarEdad(-3));
        System.out.println("Edad 150: " + v.verificarEdad(150));
    }
}
```

Explicación: La condición `edad < 0 || edad > 120` detecta los valores fuera de rango. Para 25 el método retorna `true`; para -3 y 150 imprime el mensaje de error y retorna `false`.

Salida de la ejecución

```
Edad 25: true
Edad inválida
Edad -3: false
Edad inválida
Edad 150: false
```

Ejercicio A7. Completar la clase Producto

Enunciado: Rellenar los espacios vacíos (___) para que la clase calcule el valor total del inventario de un producto.

```java
public class Producto {
    int stock;
    double precioUnitario;

    public ___ calcularValorTotal() {
        ___ stock * precioUnitario;
    }
}
```

Archivo: Producto.java

```java
public class Producto {
    int stock;
    double precioUnitario;

    public double calcularValorTotal() {
        return stock * precioUnitario;
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto();
        p.stock = 10;
        p.precioUnitario = 25.5;
        System.out.println("Valor total: " + p.calcularValorTotal());
    }
}
```

Explicación: El método lleva `double` como tipo de retorno porque calcula un valor que otra clase necesita usar, y utiliza `return` para entregar el resultado de `stock * precioUnitario`.

Salida de la ejecución

```
Valor total: 255.0
```

Ejercicio A8. Instanciación e invocación en el main

Enunciado: En el `main`: a) instanciar un objeto de `Producto` llamado `p`; b) asignarle `stock = 10` y `precioUnitario = 25.5`; c) llamar a `calcularValorTotal()`, guardar el resultado en `totalInventario` e imprimirlo.

Archivo: Producto.java

```java
public class Producto {
    int stock;
    double precioUnitario;

    public double calcularValorTotal() {
        return stock * precioUnitario;
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto();
        p.stock = 10;
        p.precioUnitario = 25.5;

        double totalInventario = p.calcularValorTotal();
        System.out.println("Total inventario: " + totalInventario);
    }
}
```

Explicación: Se crea el objeto con `new Producto()`, se asignan sus atributos con el operador punto y el valor retornado por el método se almacena en una variable `double` antes de imprimirlo: 10 × 25.5 = 255.0.

Salida de la ejecución

```
Total inventario: 255.0
```

PARTE B. Integración de clases, objetos y métodos

Ejercicio B1. Libro y Autor (objeto como atributo)

Enunciado: Crear la clase `Autor` (nombre, nacionalidad) y la clase `Libro` (titulo, anio, autor). `Libro` debe tener un atributo de tipo `Autor` y un método `mostrarInformacion()` que imprima el título, el año y el nombre del autor. En `main`, crear 2 libros.

Archivo: Autor.java

```java
public class Autor {
    String nombre;
    String nacionalidad;

    // Constructor: recibe los datos del autor
    public Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }
}
```

Archivo: Libro.java

```java
public class Libro {
    String titulo;
    int anio;
    Autor autor; // objeto como atributo

    public Libro(String titulo, int anio, Autor autor) {
        this.titulo = titulo;
        this.anio = anio;
        this.autor = autor;
    }

    public void mostrarInformacion() {
        System.out.println("Titulo: " + titulo);
        System.out.println("Anio: " + anio);
        System.out.println("Autor: " + autor.nombre + " (" + autor.nacionalidad + ")");
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Autor a1 = new Autor("Gabriel Garcia Marquez", "Colombiana");
        Autor a2 = new Autor("Jorge Icaza", "Ecuatoriana");

        Libro l1 = new Libro("Cien anios de soledad", 1967, a1);
        Libro l2 = new Libro("Huasipungo", 1934, a2);

        l1.mostrarInformacion();
        System.out.println();
        l2.mostrarInformacion();
    }
}
```

Explicación: `Libro` contiene un atributo `Autor autor`, es decir, un objeto dentro de otro objeto. Para obtener el nombre del autor se encadena el acceso: `autor.nombre`.

Salida de la ejecución

```
Titulo: Cien anios de soledad
Anio: 1967
Autor: Gabriel Garcia Marquez (Colombiana)

Titulo: Huasipungo
Anio: 1934
Autor: Jorge Icaza (Ecuatoriana)
```

Ejercicio B2. Rectángulo (firmas de métodos)

Enunciado: Crear la clase `Rectangulo` (base, altura) con `calcularArea()`, `calcularPerimetro()`, `esCuadrado()`, `obtenerDescripcion()` y `mostrar()`, usando el tipo de retorno correcto en cada firma.

Archivo: Rectangulo.java

```java
public class Rectangulo {
    double base;
    double altura;

    public Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    // Devuelve un double: el area se usa en otros calculos
    public double calcularArea() {
        return base * altura;
    }

    public double calcularPerimetro() {
        return 2 * (base + altura);
    }

    // Devuelve un boolean: es cuadrado si base y altura son iguales
    public boolean esCuadrado() {
        return base == altura;
    }

    // Devuelve un String con la descripcion
    public String obtenerDescripcion() {
        return "Rectangulo de " + base + " x " + altura;
    }

    // void: solo imprime, no devuelve nada
    public void mostrar() {
        System.out.println(obtenerDescripcion());
        System.out.println("Area: " + calcularArea());
        System.out.println("Perimetro: " + calcularPerimetro());
        System.out.println("Es cuadrado: " + esCuadrado());
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Rectangulo r1 = new Rectangulo(4.0, 6.0);
        Rectangulo r2 = new Rectangulo(5.0, 5.0);

        r1.mostrar();
        System.out.println();
        r2.mostrar();
    }
}
```

Explicación: Tipos de retorno: `double` para área y perímetro (cálculos numéricos), `boolean` para `esCuadrado()` (respuesta sí/no), `String` para la descripción y `void` para `mostrar()`, que solo imprime.

Salida de la ejecución

```
Rectangulo de 4.0 x 6.0
Area: 24.0
Perimetro: 20.0
Es cuadrado: false

Rectangulo de 5.0 x 5.0
Area: 25.0
Perimetro: 20.0
Es cuadrado: true
```

Ejercicio B3. Producto y Venta (objeto como parámetro)

Enunciado: Crear `Producto` (nombre, precio, stock) con `reducirStock(int cantidad)`, que solo actúa si hay stock suficiente (si no, imprime «Stock insuficiente.»), y `mostrarProducto()`. Crear `Venta` (cantidad) con `aplicar(Producto producto)`, que reduzca el stock y muestre el total a pagar.

Archivo: Producto.java

```java
public class Producto {
    String nombre;
    double precio;
    int stock;

    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    // Solo reduce el stock si alcanza; devuelve true si lo logro
    public boolean reducirStock(int cantidad) {
        if (cantidad <= stock) {
            stock = stock - cantidad;
            return true;
        }
        System.out.println("Stock insuficiente.");
        return false;
    }

    public void mostrarProducto() {
        System.out.println(nombre + " | Precio: $" + precio + " | Stock: " + stock);
    }
}
```

Archivo: Venta.java

```java
public class Venta {
    int cantidad;

    public Venta(int cantidad) {
        this.cantidad = cantidad;
    }

    // Objeto como parametro: trabaja sobre el Producto recibido
    public void aplicar(Producto producto) {
        if (producto.reducirStock(cantidad)) {
            double total = cantidad * producto.precio;
            System.out.println("Venta realizada. Total a pagar: $" + total);
        }
    }
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto("Mochila", 25.0, 10);
        p.mostrarProducto();

        System.out.println("\n-- Venta de 4 unidades --");
        Venta v1 = new Venta(4);
        v1.aplicar(p);
        p.mostrarProducto();

        System.out.println("\n-- Venta de 8 unidades --");
        Venta v2 = new Venta(8);
        v2.aplicar(p);
        p.mostrarProducto();
    }
}
```

Explicación: `aplicar` recibe el `Producto` como parámetro; como los objetos se pasan por referencia, el stock del producto original sí se modifica. `reducirStock` retorna `boolean` para que `Venta` sepa si debe mostrar el total. La venta de 8 unidades falla porque solo quedan 6.

Salida de la ejecución

```
Mochila | Precio: $25.0 | Stock: 10

-- Venta de 4 unidades --
Venta realizada. Total a pagar: $100.0
Mochila | Precio: $25.0 | Stock: 6

-- Venta de 8 unidades --
Stock insuficiente.
Mochila | Precio: $25.0 | Stock: 6
```

Ejercicio B4. Transferencias (objeto como parámetro)

Enunciado: Con la clase `Cuenta` (titular, saldo), agregar `transferir(Cuenta destino, double monto)`, que retire el monto de la cuenta actual y lo deposite en la destino solo si hay fondos. En `main`, crear 2 cuentas, hacer una transferencia válida y otra que falle.

Archivo: Cuenta.java

```
public class Cuenta {
    String titular;
    double saldo;

    public Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    // Objeto como parametro: la cuenta destino se recibe como argumento
    public void transferir(Cuenta destino, double monto) {
        if (monto > 0 && monto <= saldo) {
            this.saldo = this.saldo - monto;   // retira de la cuenta actual
            destino.saldo = destino.saldo + monto; // deposita en la destino
            System.out.println("Transferencia exitosa de $" + monto
                    + " de " + titular + " a " + destino.titular);
        } else {
            System.out.println("Transferencia fallida: fondos insuficientes en la cuenta de
" + titular);
        }
    }

    public void mostrarSaldo() {
        System.out.println(titular + " : $" + saldo);
    }
}
```

Archivo: Main.java

```
public class Main {
    public static void main(String[] args) {
        Cuenta c1 = new Cuenta("Carlos", 500);
        Cuenta c2 = new Cuenta("Laura", 100);

        System.out.println("Saldos iniciales");
        c1.mostrarSaldo();
        c2.mostrarSaldo();

        System.out.println("\nTransferencia valida (200)");
        c1.transferir(c2, 200);

        System.out.println("\nTransferencia que falla (1000)");
        c1.transferir(c2, 1000);

        System.out.println("\nSaldos finales");
        c1.mostrarSaldo();
        c2.mostrarSaldo();
    }
}
```

Explicación: El método modifica dos objetos: la cuenta actual (`this`) y la cuenta `destino` recibida como parámetro. La condición `monto <= saldo` evita saldos negativos; la transferencia de 1000 falla porque Carlos solo tiene 300.

Salida de la ejecución

```
Saldos iniciales
Carlos : $500.0
Laura : $100.0

Transferencia valida (200)
Transferencia exitosa de $200.0 de Carlos a Laura

Transferencia que falla (1000)
Transferencia fallida: fondos insuficientes en la cuenta de Carlos

Saldos finales
Carlos : $300.0
Laura : $300.0
```

Ejercicio B5. Biblioteca (colaboración entre clases)

Enunciado: Crear `Usuario` (nombre, cedula), `Libro` (titulo, disponible, con `prestar()` y `devolver()`) y `Prestamo` (usuario, libro) con `registrar()`, que intenta prestar el libro y muestra un mensaje según el resultado. En `main`, hacer que dos usuarios intenten pedir el mismo libro.

Archivo: Usuario.java

```java
public class Usuario {
    String nombre;
    String cedula;

    public Usuario(String nombre, String cedula) {
        this.nombre = nombre;
        this.cedula = cedula;
    }
}
```

Archivo: Libro.java

```java
public class Libro {
    String titulo;
    boolean disponible;

    public Libro(String titulo) {
        this.titulo = titulo;
        this.disponible = true;
    }

    // Presta el libro solo si esta disponible
    public boolean prestar() {
        if (disponible) {
            disponible = false;
            return true;
        }
        return false;
    }

    public void devolver() {
        disponible = true;
    }
}
```

Archivo: Prestamo.java

```
public class Prestamo {
    Usuario usuario; // colabora con Usuario
    Libro libro;     // colabora con Libro

    public Prestamo(Usuario usuario, Libro libro) {
        this.usuario = usuario;
        this.libro = libro;
    }

    public void registrar() {
        if (libro.prestar()) {
            System.out.println(usuario.nombre + " (CI " + usuario.cedula + ") obtuvo el
libro \""
                    + libro.titulo + "\".");
        } else {
            System.out.println(usuario.nombre + " (CI " + usuario.cedula + ") no pudo
obtener \""
                    + libro.titulo + "\": ya esta prestado.");
        }
    }
}
```

Archivo: Main.java

```
public class Main {
    public static void main(String[] args) {
        Usuario u1 = new Usuario("Maria", "1801234567");
        Usuario u2 = new Usuario("Juan", "1807654321");
        Libro libro = new Libro("Don Quijote");

        // Dos usuarios intentan pedir el mismo libro
        Prestamo p1 = new Prestamo(u1, libro);
        Prestamo p2 = new Prestamo(u2, libro);

        p1.registrar();
        p2.registrar();

        System.out.println("\nMaria devuelve el libro.");
        libro.devolver();
        p2.registrar();
    }
}
```

Explicación: `Prestamo` colabora con `Usuario` y `Libro` mediante sus atributos. `prestar()` retorna `true` y marca el libro como no disponible solo si estaba disponible; por eso el segundo usuario no puede obtenerlo hasta que se ejecuta `devolver()`.

Salida de la ejecución

```
Maria (CI 1801234567) obtuvo el libro "Don Quijote".
Juan (CI 1807654321) no pudo obtener "Don Quijote": ya esta prestado.

Maria devuelve el libro.
Juan (CI 1807654321) obtuvo el libro "Don Quijote".
```

PARTE C. Introducción a la programación orientada a objetos

Ejercicio C1. Notas del curso

Enunciado: Crear la clase `Estudiante` (nombre, nota1, nota2, nota3, promedio). En `main`, crear 5 instancias, calcular el promedio de cada una e imprimirlas con el formato *nombre : promedio*, agregando Aprobado (7 o más), Supletorio (5 a 6.99) o Reprobado (menos de 5). Al final, imprimir cuántos aprobaron.

Archivo: Estudiante.java

```java
public class Estudiante {
    String nombre;
    double nota1;
    double nota2;
    double nota3;
    double promedio;
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Estudiante e1 = new Estudiante();
        e1.nombre = "Ana";
        e1.nota1 = 8;
        e1.nota2 = 9;
        e1.nota3 = 7;
        e1.promedio = (e1.nota1 + e1.nota2 + e1.nota3) / 3;

        Estudiante e2 = new Estudiante();
        e2.nombre = "Luis";
        e2.nota1 = 6;
        e2.nota2 = 5;
        e2.nota3 = 7;
        e2.promedio = (e2.nota1 + e2.nota2 + e2.nota3) / 3;

        Estudiante e3 = new Estudiante();
        e3.nombre = "Maria";
        e3.nota1 = 4;
        e3.nota2 = 3;
        e3.nota3 = 5;
        e3.promedio = (e3.nota1 + e3.nota2 + e3.nota3) / 3;

        Estudiante e4 = new Estudiante();
        e4.nombre = "Pedro";
        e4.nota1 = 10;
        e4.nota2 = 9;
        e4.nota3 = 8;
        e4.promedio = (e4.nota1 + e4.nota2 + e4.nota3) / 3;

        Estudiante e5 = new Estudiante();
        e5.nombre = "Sofia";
        e5.nota1 = 7;
        e5.nota2 = 6;
        e5.nota3 = 8;
        e5.promedio = (e5.nota1 + e5.nota2 + e5.nota3) / 3;

        int aprobados = 0;

        System.out.println("NOTAS DEL CURSO");

        if (e1.promedio >= 7) {
            System.out.println(e1.nombre + " : " + e1.promedio + " Aprobado");
            aprobados = aprobados + 1;
        } else if (e1.promedio >= 5) {
            System.out.println(e1.nombre + " : " + e1.promedio + " Supletorio");
        } else {
            System.out.println(e1.nombre + " : " + e1.promedio + " Reprobado");
        }

        if (e2.promedio >= 7) {
            System.out.println(e2.nombre + " : " + e2.promedio + " Aprobado");
            aprobados = aprobados + 1;
        } else if (e2.promedio >= 5) {
            System.out.println(e2.nombre + " : " + e2.promedio + " Supletorio");
```

```
        } else {
            System.out.println(e2.nombre + " : " + e2.promedio + " Reprobado");
        }

        if (e3.promedio >= 7) {
            System.out.println(e3.nombre + " : " + e3.promedio + " Aprobado");
            aprobados = aprobados + 1;
        } else if (e3.promedio >= 5) {
            System.out.println(e3.nombre + " : " + e3.promedio + " Supletorio");
        } else {
            System.out.println(e3.nombre + " : " + e3.promedio + " Reprobado");
        }

        if (e4.promedio >= 7) {
            System.out.println(e4.nombre + " : " + e4.promedio + " Aprobado");
            aprobados = aprobados + 1;
        } else if (e4.promedio >= 5) {
            System.out.println(e4.nombre + " : " + e4.promedio + " Supletorio");
        } else {
            System.out.println(e4.nombre + " : " + e4.promedio + " Reprobado");
        }

        if (e5.promedio >= 7) {
            System.out.println(e5.nombre + " : " + e5.promedio + " Aprobado");
            aprobados = aprobados + 1;
        } else if (e5.promedio >= 5) {
            System.out.println(e5.nombre + " : " + e5.promedio + " Supletorio");
        } else {
            System.out.println(e5.nombre + " : " + e5.promedio + " Reprobado");
        }

        System.out.println("Cantidad de estudiantes que aprobaron: " + aprobados);
    }
}
```

Análisis: Se crean cinco estudiantes y se calcula el promedio sumando las tres notas y dividiendo para 3. Con `if`, `else if` y `else` se decide la condición, y el contador `aprobados` aumenta cada vez que alguien aprueba.

Salida de la ejecución

```
NOTAS DEL CURSO
Ana : 8.0 Aprobado
Luis : 6.0 Supletorio
Maria : 4.0 Reprobado
Pedro : 9.0 Aprobado
Sofia : 7.0 Aprobado
Cantidad de estudiantes que aprobaron: 3
```

Ejercicio C2. Inventario de una tienda

Enunciado: Crear `Producto` (nombre, precio, stock), crear 4 productos e imprimirlos con el formato *nombre : precio*. Simular la venta de 3 unidades de dos de ellos, mostrar «Reponer» junto a cada producto con menos de 5 unidades e imprimir el valor total del inventario.

Archivo: Producto.java

```
public class Producto {
    String nombre;
    double precio;
    int stock;
}
```

Archivo: Main.java

```
public class Main {
    public static void main(String[] args) {
```

```
        Producto p1 = new Producto();
        p1.nombre = "Cuaderno";
        p1.precio = 1.5;
        p1.stock = 7;

        Producto p2 = new Producto();
        p2.nombre = "Lapiz";
        p2.precio = 0.5;
        p2.stock = 20;

        Producto p3 = new Producto();
        p3.nombre = "Mochila";
        p3.precio = 25;
        p3.stock = 6;

        Producto p4 = new Producto();
        p4.nombre = "Borrador";
        p4.precio = 0.75;
        p4.stock = 8;

        System.out.println("PRODUCTOS");
        System.out.println(p1.nombre + " : " + p1.precio);
        System.out.println(p2.nombre + " : " + p2.precio);
        System.out.println(p3.nombre + " : " + p3.precio);
        System.out.println(p4.nombre + " : " + p4.precio);

        // Venta de 3 unidades de dos productos
        p1.stock = p1.stock - 3;
        p3.stock = p3.stock - 3;

        System.out.println();
        System.out.println("DESPUES DE LA VENTA");

        if (p1.stock < 5) {
            System.out.println(p1.nombre + " : " + p1.precio + " Reponer");
        } else {
            System.out.println(p1.nombre + " : " + p1.precio);
        }

        if (p2.stock < 5) {
            System.out.println(p2.nombre + " : " + p2.precio + " Reponer");
        } else {
            System.out.println(p2.nombre + " : " + p2.precio);
        }

        if (p3.stock < 5) {
            System.out.println(p3.nombre + " : " + p3.precio + " Reponer");
        } else {
            System.out.println(p3.nombre + " : " + p3.precio);
        }

        if (p4.stock < 5) {
            System.out.println(p4.nombre + " : " + p4.precio + " Reponer");
        } else {
            System.out.println(p4.nombre + " : " + p4.precio);
        }

        double total = p1.precio * p1.stock + p2.precio * p2.stock
                + p3.precio * p3.stock + p4.precio * p4.stock;

        System.out.println();
        System.out.println("Valor total del inventario: " + total);
    }
}
```

Análisis: La venta se simula restando 3 al stock del cuaderno y de la mochila. Luego un `if` agrega la palabra «Reponer» cuando el stock es menor que 5. El valor total se obtiene multiplicando precio por stock de cada producto y sumando todo.

Salida de la ejecución

```
PRODUCTOS
Cuaderno : 1.5
Lapiz : 0.5
Mochila : 25.0
Borrador : 0.75

DESPUES DE LA VENTA
Cuaderno : 1.5 Reponer
Lapiz : 0.5
Mochila : 25.0 Reponer
Borrador : 0.75

Valor total del inventario: 97.0
```

Ejercicio C3. Veterinaria

Enunciado: Crear `Mascota` (nombre, especie, edad, peso), crear 3 mascotas e imprimirlas con el formato *nombre : especie : peso*, mostrando «Revisar peso» si pesan más de 20 kg. Luego escribir `Mascota m4 = m1;`, sumar 1 a `m4.edad`, imprimir `m1.edad` y explicar en un comentario por qué cambió.

Archivo: Mascota.java

```java
public class Mascota {
    String nombre;
    String especie;
    int edad;
    double peso;
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Mascota m1 = new Mascota();
        m1.nombre = "Firulais";
        m1.especie = "Perro";
        m1.edad = 5;
        m1.peso = 25.5;

        Mascota m2 = new Mascota();
        m2.nombre = "Michi";
        m2.especie = "Gato";
        m2.edad = 3;
        m2.peso = 4.2;

        Mascota m3 = new Mascota();
        m3.nombre = "Rocky";
        m3.especie = "Perro";
        m3.edad = 2;
        m3.peso = 12;

        System.out.println("MASCOTAS");

        if (m1.peso > 20) {
            System.out.println(m1.nombre + " : " + m1.especie + " : " + m1.peso + " Revisar
peso");
        } else {
            System.out.println(m1.nombre + " : " + m1.especie + " : " + m1.peso);
        }

        if (m2.peso > 20) {
```

```
            System.out.println(m2.nombre + " : " + m2.especie + " : " + m2.peso + " Revisar
peso");
        } else {
            System.out.println(m2.nombre + " : " + m2.especie + " : " + m2.peso);
        }

        if (m3.peso > 20) {
            System.out.println(m3.nombre + " : " + m3.especie + " : " + m3.peso + " Revisar
peso");
        } else {
            System.out.println(m3.nombre + " : " + m3.especie + " : " + m3.peso);
        }

        System.out.println();
        System.out.println("Edad de m1 antes: " + m1.edad);

        Mascota m4 = m1;
        m4.edad = m4.edad + 1;

        System.out.println("Edad de m1 despues: " + m1.edad);

        // Explicacion: la edad de m1 cambio porque cuando escribo Mascota m4 = m1;
        // no se crea una mascota nueva. Solo se copia la referencia, o sea, la
        // direccion donde esta guardado el objeto. Entonces m1 y m4 son dos
        // nombres para el mismo objeto. Si cambio la edad usando m4, tambien
        // cambia en m1 porque es la misma mascota.
    }
}
```

Análisis: `Mascota m4 = m1;` no crea un objeto nuevo: copia la referencia. `m1` y `m4` son dos nombres para el mismo objeto, por eso la edad de `m1` cambia de 5 a 6.

Salida de la ejecución

```
MASCOTAS
Firulais : Perro : 25.5 Revisar peso
Michi : Gato : 4.2
Rocky : Perro : 12.0

Edad de m1 antes: 5
Edad de m1 despues: 6
```

Ejercicio C4. Cuenta de ahorro

Enunciado: Crear `Cuenta` (titular, saldo) con dos cuentas, una con 200 y otra con 350. Con un `while`, depositar 50 por mes en cada una hasta llegar a 500, imprimiendo cada mes con el formato *titular : saldo*, y al final cuántos meses necesitó cada una (6 y 3).

Archivo: Cuenta.java

```
public class Cuenta {
    String titular;
    double saldo;
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Cuenta c1 = new Cuenta();
        c1.titular = "Carlos";
        c1.saldo = 200;

        Cuenta c2 = new Cuenta();
        c2.titular = "Laura";
        c2.saldo = 350;

        int meses1 = 0;
        int meses2 = 0;

        System.out.println("CUENTA DE " + c1.titular);
        while (c1.saldo < 500) {
            c1.saldo = c1.saldo + 50;
            meses1 = meses1 + 1;
            System.out.println(c1.titular + " : " + c1.saldo);
        }

        System.out.println();

        System.out.println("CUENTA DE " + c2.titular);
        while (c2.saldo < 500) {
            c2.saldo = c2.saldo + 50;
            meses2 = meses2 + 1;
            System.out.println(c2.titular + " : " + c2.saldo);
        }

        System.out.println();
        System.out.println(c1.titular + " necesito " + meses1 + " meses");
        System.out.println(c2.titular + " necesito " + meses2 + " meses");
    }
}
```

Análisis: El `while` se repite mientras el saldo sea menor que 500. En cada vuelta se suman 50 al saldo, se aumenta en 1 el contador de meses y se imprime el saldo. Carlos necesita 6 meses y Laura 3.

Salida de la ejecución

```
CUENTA DE Carlos
Carlos : 250.0
Carlos : 300.0
Carlos : 350.0
Carlos : 400.0
Carlos : 450.0
Carlos : 500.0

CUENTA DE Laura
Laura : 400.0
Laura : 450.0
Laura : 500.0

Carlos necesito 6 meses
Laura necesito 3 meses
```

Ejercicio C5. Biblioteca

Enunciado: Crear `Libro` (titulo, autor, disponible) y `Socio` (nombre, librosPrestados). Crear 4 libros y 2 socios; el primer socio presta 2 libros y el segundo 1 (disponible = false y aumentar librosPrestados). Imprimir cada libro como *titulo : Disponible/Prestado*, cada socio como *nombre : librosPrestados* y cuántos libros quedan disponibles.

Archivo: Libro.java

```java
public class Libro {
    String titulo;
    String autor;
    boolean disponible;
}
```

Archivo: Socio.java

```java
public class Socio {
    String nombre;
    int librosPrestados;
}
```

Archivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Libro l1 = new Libro();
        l1.titulo = "Don Quijote";
        l1.autor = "Cervantes";
        l1.disponible = true;

        Libro l2 = new Libro();
        l2.titulo = "El Principito";
        l2.autor = "Saint-Exupery";
        l2.disponible = true;

        Libro l3 = new Libro();
        l3.titulo = "Rayuela";
        l3.autor = "Cortazar";
        l3.disponible = true;

        Libro l4 = new Libro();
        l4.titulo = "Ficciones";
        l4.autor = "Borges";
        l4.disponible = true;

        Socio s1 = new Socio();
        s1.nombre = "Maria";
        s1.librosPrestados = 0;

        Socio s2 = new Socio();
        s2.nombre = "Juan";
        s2.librosPrestados = 0;

        l1.disponible = false;
        s1.librosPrestados = s1.librosPrestados + 1;
        l2.disponible = false;
        s1.librosPrestados = s1.librosPrestados + 1;

        l3.disponible = false;
        s2.librosPrestados = s2.librosPrestados + 1;

        int disponibles = 0;

        System.out.println("LIBROS");

        if (l1.disponible) {
            System.out.println(l1.titulo + " : Disponible");
            disponibles = disponibles + 1;
        } else {
            System.out.println(l1.titulo + " : Prestado");
        }

        if (l2.disponible) {
            System.out.println(l2.titulo + " : Disponible");
```

```
            disponibles = disponibles + 1;
        } else {
            System.out.println(l2.titulo + " : Prestado");
        }

        if (l3.disponible) {
            System.out.println(l3.titulo + " : Disponible");
            disponibles = disponibles + 1;
        } else {
            System.out.println(l3.titulo + " : Prestado");
        }

        if (l4.disponible) {
            System.out.println(l4.titulo + " : Disponible");
            disponibles = disponibles + 1;
        } else {
            System.out.println(l4.titulo + " : Prestado");
        }

        System.out.println();
        System.out.println("SOCIOS");
        System.out.println(s1.nombre + " : " + s1.librosPrestados);
        System.out.println(s2.nombre + " : " + s2.librosPrestados);

        System.out.println();
        System.out.println("Libros disponibles: " + disponibles);
    }
}
```

Análisis: El atributo `disponible` es de tipo `boolean`. Al prestar un libro se pone en `false` y se suma 1 a `librosPrestados` del socio. Al imprimir, un `if` decide entre «Disponible» y «Prestado» y una variable cuenta los libros que siguen disponibles.

Salida de la ejecución

```
LIBROS
Don Quijote : Prestado
El Principito : Prestado
Rayuela : Prestado
Ficciones : Disponible

SOCIOS
Maria : 2
Juan : 1

Libros disponibles: 1
```

➢ **Pruebas de escritorio:**

Ej.	Entrada	Proceso	Resultado esperado	Resultado obtenido
A4	numero = 10	modificar(numero) suma 5 a la copia	Dentro: 15 · Después: 10	Dentro: 15 · Después: 10
A6	edad = 25	25 no es < 0 ni > 120	true	true
A6	edad = -3	-3 < 0 → «Edad inválida»	false	false
A6	edad = 150	150 > 120 → «Edad inválida»	false	false
B3	stock = 10, venta de 4	4 ≤ 10 → stock = 6; total = 4 × 25	Total $100.0	Total $100.0
B3	stock = 6, venta de 8	8 > 6 → no reduce	Stock insuficiente.	Stock insuficiente.
B4	Carlos 500, Laura 100; monto 200	200 ≤ 500 → Carlos 300, Laura 300	Transferencia exitosa	Transferencia exitosa
B4	Carlos 300; monto 1000	1000 > 300 → no transfiere	Transferencia fallida	Transferencia fallida
B5	Dos usuarios piden «Don Quijote»	El primero: disponible → false; el segundo: ya no disponible	El 2.º no obtiene el libro	El 2.º no obtiene el libro
C3	m1.edad = 5; m4 = m1; m4.edad + 1	m1 y m4 apuntan al mismo objeto	m1.edad = 6	m1.edad = 6
C4	Carlos 200 → 500 (+50)	200→250→…→500 (6 vueltas)	6 meses	6 meses
C4	Laura 350 → 500 (+50)	350→400→450→500 (3 vueltas)	3 meses	3 meses

2.5 Resultados obtenidos

Los 18 ejercicios compilaron y se ejecutaron sin errores en Java, y la salida obtenida coincide con el resultado esperado de las pruebas de escritorio. Se comprobó que los métodos `void` ejecutan acciones, que los métodos con tipo de retorno entregan valores al `main`, que los tipos primitivos se pasan por valor y que los objetos permiten compartir y modificar datos entre clases.

Ej.	Tema	Clases	Resultado
A1	Tipos de retorno y error de compilación	Main	int vs void; return obligatorio
A2	Método void (Bienvenida)	Bienvenida, Main	Saludo con nombre y edad
A3	Parámetros vs. argumentos	Main	Resultado: 8
A4	Paso por valor	Main	Dentro 15 / después 10
A5	Múltiples parámetros (Calculadora)	Calculadora, Main	Promedio: 16.33
A6	Validación de datos (verificarEdad)	Validador, Main	true / false según el rango
A7	Completar la clase Producto	Producto, Main	Valor total: 255.0
A8	Instanciación e invocación en el main	Producto, Main	Total inventario: 255.0
B1	Libro y Autor	Autor, Libro, Main	Objeto como atributo
B2	Rectángulo	Rectangulo, Main	Área, perímetro y cuadrado
B3	Producto y Venta	Producto, Venta, Main	Venta válida y «Stock insuficiente.»
B4	Transferencias	Cuenta, Main	Transferencia válida y fallida
B5	Biblioteca (Usuario, Libro, Préstamo)	Usuario, Libro, Prestamo, Main	El segundo usuario no puede prestar
C1	Notas del curso	Estudiante, Main	3 aprobados
C2	Inventario de una tienda	Producto, Main	Valor total: 97.0
C3	Veterinaria	Mascota, Main	m1.edad pasa de 5 a 6
C4	Cuenta de ahorro	Cuenta, Main	Carlos 6 meses, Laura 3
C5	Biblioteca	Libro, Socio, Main	1 libro disponible

2.6 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☒ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☒ Adaptabilidad
☒ Responsabilidad

2.7 Conclusiones

- Se logró desarrollar programas en Java con clases, objetos y métodos, lo que permitió organizar el código en unidades reutilizables y disminuir errores.
- La práctica permitió distinguir cuándo un método debe ser `void` y cuándo debe retornar un valor con `return`, y comprender que Java pasa los tipos primitivos por valor, mientras que dos variables de objeto pueden referirse al mismo objeto, como ocurrió con `m4 = m1`.
- La colaboración entre clases (`Libro–Autor`, `Producto–Venta`, `Usuario–Libro–Prestamo`) demostró que pasar objetos como atributos y parámetros permite resolver problemas más cercanos a situaciones reales [1], [2].

2.8 Recomendaciones

- Revisar el material proporcionado y la bibliografía antes de elaborar las tareas, y acudir a las tutorías académicas cuando existan dudas.
- Validar siempre los datos antes de procesarlos (edades fuera de rango, stock insuficiente, fondos insuficientes) para evitar resultados incorrectos.
- Comentar el código y probar cada clase con distintos valores de entrada, comparando la salida con una prueba de escritorio.

2.9 Bibliografía

[1] M. I. Barquilla Galeano, *Java programación: teoría y ejemplos*, 1.ª ed. Madrid, España: RA-MA Editorial, 2025.

[2] F. Blasco, *Programación orientada a objetos en Java*, 1.ª ed. Bogotá, Colombia: Ediciones de la U, 2019.

[3] Oracle, “Java Documentation,” Oracle. [En línea]. Disponible en: https://docs.oracle.com/javase/. [Accedido: 05-nov-2026].

[4] Microsoft, “Documentation for Visual Studio Code,” Visual Studio Code. [En línea]. Disponible en: https://code.visualstudio.com/docs. [Accedido: 05-nov-2026].

2.10 Anexos

Ilustración 1. Estructura del proyecto desarrollado

```
ejercicios-ape-java/
├── parteA/   ejercicio1 … ejercicio8   (métodos, parámetros, retorno)
├── parteB/   ejercicio1 … ejercicio5   (integración de clases)
└── parteC/   ejercicio1 … ejercicio5   (introducción a la POO)

Compilar y ejecutar (dentro de cada carpeta):
  javac -encoding UTF-8 *.java
  java Main
```

LINK GITHUB: https://github.com/Anthony-GTX/MetodosEnJavaGrupo5